Release Date: 06/05/2026 — ProSoft

EasySave v2.0

Technical Documentation

1. Overview

EasySave is a backup automation tool developed as part of the ProSoft software suite. Version 2.0 adds a WPF graphical interface using the MVVM architecture pattern, unlimited backup jobs, XOR file encryption via CryptoSoft, and business software detection. The software targets .NET 8.0 on Windows and relies on the EasyLog.dll logging library.

2. Architecture

2.1 Projects

Project	Type	Purpose
EasySave	Console app (.exe)	Interactive menu, CLI mode, job management (v1.1)
EasySave.GUI	WPF app (.exe)	Graphical interface using MVVM (v2.0)
EasyLog	Class library (.dll)	Daily log (JSON/XML) and real-time state file
CryptoSoft	Console app (.exe)	XOR file encryption, called by EasySave as a subprocess

2.2 EasySave Class Overview

File	Role
Program.cs	Entry point. Interactive menu and CLI argument path.
BackupManager.cs	Runs jobs. Picks strategy, handles business software check.
Models/BackupJob.cs	Four-field job model: name, source, target, type.
Models/BackupType.cs	Enum: Full and Differential.
Models/AppSettings.cs	Settings model: LogFormat, Language, CryptoSoftPath, lists.
Services/IBackupStrategy.cs	Interface implemented by both strategy classes.
Services/BackupStrategyBase.cs	Abstract base: file transfer, encryption, logging, state updates.
Services/FullBackupStrategy.cs	Copies every file on every run.
Services/DifferentialBackupStrategy.cs	Copies only new or modified files.
Services/CryptoSoftRunner.cs	Launches CryptoSoft.exe as a subprocess for encryption.
Services/BusinessSoftwareDetector.cs	Checks running processes against the configured list.
Config/ConfigManager.cs	Reads and writes config.json and settings.json.
Languages/LanguageManager.cs	English, French and Spanish string dictionaries.

2.3 EasySave.GUI Class Overview

File	Role
ViewModels/ViewModelBase.cs	INotifyPropertyChanged base class for all ViewModels.

ViewModels/RelayCommand.cs	ICommand implementation for MVVM button bindings.
ViewModels/JobViewModel.cs	Wraps BackupJob for display and editing in the UI.
ViewModels/MainViewModel.cs	Main logic: job list, run commands, status message.
ViewModels/SettingsViewModel.cs	Settings window logic: collections, commands, model conversion.
Views/MainWindow.xaml	Main window layout with job table and action buttons.
Views/JobEditWindow.xaml	Add/edit job dialog.
Views/SettingsWindow.xaml	Settings dialog with all configurable fields.

2.4 EasyLog Class Overview

File	Role
LogEntry.cs	Seven-field model including EncryptionTime.
StateEntry.cs	Eight-field model for the live state of one job.
DailyLogger.cs	Appends entries to daily JSON or XML log file.
StateLogger.cs	Keeps in-memory job states, rewrites state.json after every file.
LogFormat.cs	Enum: Json and Xml.

2.5 Architecture Patterns

Strategy Pattern (v1.0+): BackupManager selects FullBackupStrategy or DifferentialBackupStrategy at runtime based on the job type. Adding new backup modes requires only a new class implementing IBackupStrategy.

MVVM Pattern (v2.0): The WPF GUI follows Model-View-ViewModel. Views bind to ViewModels via data binding. ViewModelBase implements INotifyPropertyChanged. RelayCommand implements ICommand for button actions. The View never directly accesses business logic.

3. Features

3.1 Backup Jobs

- Unlimited named jobs, persisted in config.json across restarts.
- Each job stores name, source directory, target directory, and type.
- Source and target support local disks, external drives, and UNC network paths.

3.2 Backup Types

Type	Behaviour
Full	Every source file is copied to target on every run, overwriting existing files.
Differential	Only files absent in target or with a newer modification date are transferred.

3.3 File Encryption (CryptoSoft)

- After each file is copied, BackupStrategyBase checks if the extension is in EncryptedExtensions.
- If matched, CryptoSoftRunner launches CryptoSoft.exe with the target file path as argument.
- CryptoSoft applies XOR encryption with key { 0x4A, 0x6F, 0x79, 0x21 } and overwrites the file.
- CryptoSoft returns elapsed milliseconds as its exit code.
- EncryptionTime in the log: 0 = not encrypted, >0 = success ms, -1 = error.

3.4 Business Software Detection

• BusinessSoftwareDetector.IsRunning() checks Process.GetProcessesByName() for each configured name.

• Checked once before a job starts and once before each file during execution.

• If detected, an InvalidOperationException is thrown and shown as an error status in the GUI.

• The interruption is logged with FileTransferTime = -1.

3.5 Log Format

Format	Extension	Notes
Json (default)	YYYY-MM-DD.json	Backward-compatible with v1.0 and v1.1.
Xml	YYYY-MM-DD.xml	Same fields, XML structure. Configurable in Settings.

4. Log Files

4.1 Daily Log — YYYY-MM-DD.json / .xml

Location: %APPDATA%\EasySave\Logs\YYYY-MM-DD.{json|xml}

Field	Type	Description
Name	string	Name of the backup job.
FileSource	string	Full path of the source file.
FileTarget	string	Full path of the destination file.
FileSize	long	File size in bytes at time of transfer.
FileTransferTime	double	Duration in ms. Negative = copy error.
EncryptionTime	double	0 = no encryption, >0 = ms elapsed, -1 = error.
Time	string	Timestamp: dd/MM/yyyy HH:mm:ss.

4.2 State File — state.json

Location: %APPDATA%\EasySave\State\state.json — always JSON.

Field	Type	Description
Name	string	Job name, matches config.json.
State	string	One of: ACTIVE, END.
SourceFilePath	string	File currently being copied.
TargetFilePath	string	Destination of the file currently being copied.
TotalFilesToCopy	int	Number of files selected for this run.
TotalFilesSize	long	Combined size in bytes.
NbFilesLeftToDo	int	Files remaining.
Progression	int	Completion percentage (0-100).

5. Configuration & File Locations

File	Path
Job configuration	%APPDATA%\EasySave\config.json

App settings	%APPDATA%\EasySave\settings.json
Daily log	%APPDATA%\EasySave\Logs\YYYY-MM-DD.{json|xml}
State file	%APPDATA%\EasySave\State\state.json

6. Minimum Requirements

Component	Requirement
Operating system	Windows 10 / Windows Server 2016 or later
Runtime	.NET 8.0 Runtime
Disk space	10 MB for the application + sufficient space on target drive
Permissions	Read on source directory, write on target directory